TITLE

Project Description:

- In a single paragraph explain about the project.

Scenario:

System Requirements:

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above)**: Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 👉 Download Node.js
- **npm (v8 or above)**: A package manager required to install dependencies for React and related libraries.
- **React.js**: A JavaScript library for building dynamic and responsive user interfaces.
- **Browser**: Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Express.js**: A lightweight web framework for building RESTful APIs.
- **MongoDB**: A NoSQL database used to store structured and unstructured data related to farms, users, and resources. 👉 Download MongoDB
- **Postman**: Tool for testing APIs during development.

- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.
- **Git & GitHub**: For version control and collaborative development.

2. Hardware Requirements

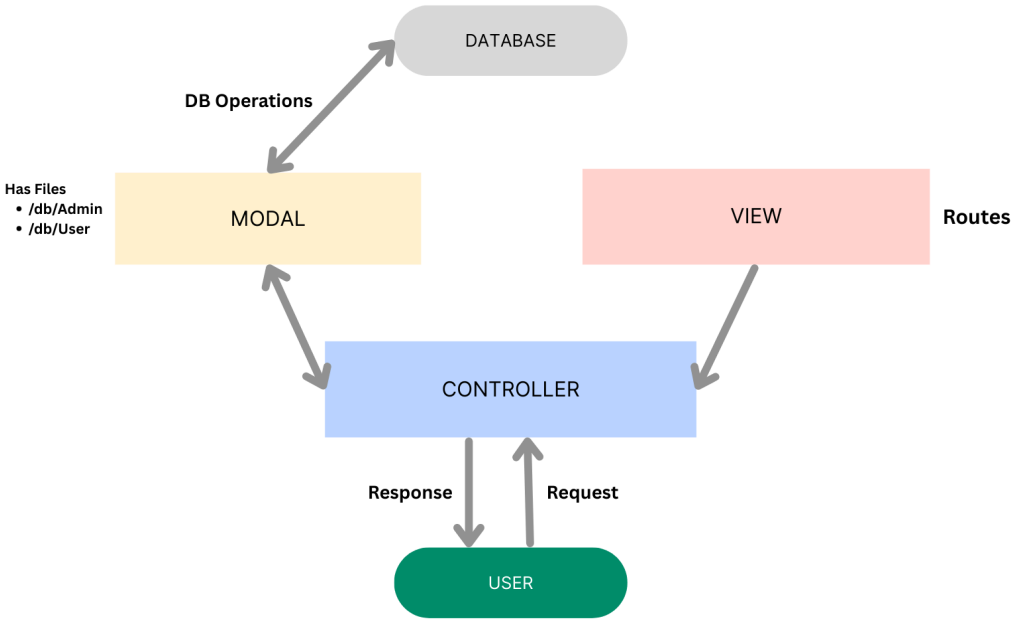
Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM**: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage**: At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display**: 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

Project Architecture:

- **Technical Architecture:**
  - With explanation
- **ER Diagram:**
  - With explanation
- **Features**
- **Roles and responsibilities**
- **User Flow**

- **MVC Pattern**



The Agri-Tech backend application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include

validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- **Separation of Concerns**: Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability**: New features can be added easily by creating new routes, controllers, and models.
- **Reusability**: Logic in controllers and models can be reused across multiple parts of the application.
- **Testing**: Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly**: Multiple developers can work simultaneously on different layers without conflict.

Project Setup And Configuration.

- ### Creating project folder

  1. Create a new folder with your <project name>.
  2. Inside that folder create two new folders.
  3. Name one as **Client**.
  4. Name another one as **Server**.
  5. Now open that folder in VS Code.

- ### Client setup (installing react app)

- Open the Client folder in the terminal of VScode.
  - npm create vite@latest . -- --template react
- Select React framework from the given options.
  - Select a framework:
  - │ React
- Select JavaScript variant from the given options.
  - Select a variant:
  - │ JavaScript
- Now lets navigate to the client folder by giving the following command.
  - cd client
- To install all the packages run the following command.
  - npm install
- To start the React server type the following command.
  - npm run dev

- **Server setup (npm init)**
  - Open Server folder in terminal of VScode.
    - npm init -y
  - Create files:
    - `server.js`
  - Create folders:
    - models
    - controllers
    - routes

Backend Development:

- Reference video with voiceover or screen shots for backend code

  Note:- Proper Explanation needs to be there for each and every third-party-API consumed.

Database Development

- Reference video with voiceover or screen shots for Database code

Front-End Development.

- Reference video with voiceover or screen shots for frontend code

  Note:- Proper Explanation needs to be there for each and every third-party-API consumed.

Project Execution.

- **Steps for Project execution**

  Step 1: Download and Open the Project

  1. First, clone or download the project folder (named `ucab-app`) from the repository or your local source.
  2. Once downloaded, open the project folder in VS Code or any code editor of your choice.

  You will see two main subfolders:

  - **`client`** → contains the frontend (React) code
    `server` → contains the backend (Node.js + Express) code

Step 2: Set Up the Frontend (React App):

a) Open a terminal and navigate into the client folder:

cd client

b) Install all the required frontend packages:

npm install

c) Once installation is done, start the React development server:
npm run dev

d) The app should now be running on:
http://localhost:5173

Step 3: Set Up the Backend (Express Server)

a) Open a new terminal tab/window or split the terminal.

b) Navigate into the server folder:
cd ../server

c) Install all backend dependencies (like Express, Mongoose, etc.):
npm install

Step 4: Configure Environment Variables

a) **Inside the** server f**older, create a new file named .**env **(no file extension).**

b) **In that** .env **file, add your MongoDB connection string:**
MONGO_URI=mongodb://localhost:27017/ucab

Step 5: Start the Backend Server:

a) Inside the same server folder, run the backend server using**:**

nodemon index.js

b) The server should start on:
http://localhost:8000

- **Output ScreenShots with explanation:**
- **Demo video link (This link will be given by Shivani mam)**
- **Code drive link (This link will be given by Shivani mam)**

Note:-(Mandatory Functionality to be there in Each and Every Project, With out this features no projects will be marked as done)

1)MVC Architecture in Backend

2)JWT

3)React-Vite from now Onwards(04-06-2025)

4)Protected Routes

5)Better UI with Tailwind CSS or Bootstrap